

Exercício: projeto de conjuntos de teste mínimos para uma implementação de Tabela Hash

Neste exercício, você deverá elaborar quatro conjuntos de casos de teste para o programa de Tabela Hash fornecido pelo professor.

O objetivo não é apenas atingir cobertura, mas também construir conjuntos sem casos de teste supérfluos. Isto é, em cada conjunto entregue, nenhum caso de teste pode ser removido sem que a cobertura obtida para o critério-alvo seja reduzida.

Em outras palavras, cada conjunto deve ser:

- adequado ao critério solicitado; e
- essencial em relação a esse critério.

Arquivo a ser testado

O arquivo a ser testado é o arquivo da implementação de Tabela Hash fornecido pelo professor.

Para fins de organização, assuma que ele deve ser importado pelos testes como o módulo principal da atividade.

Objetivos

Você deverá construir quatro conjuntos independentes de testes:

1. um conjunto adequado ao critério **cobertura de arestas**;
2. um conjunto adequado ao critério **MC/DC**;
3. um conjunto adequado ao critério **edge-pair**;
4. um conjunto adequado ao critério **prime paths**.

Cada conjunto deve ser criado e justificado separadamente.

Arquivos de teste a serem entregues

Os nomes dos arquivos de teste devem seguir o padrão abaixo:

- `test_hash_desvios.py`
Conjunto adequado ao critério de **cobertura de arestas**.
- `test_hash_mcdc.py`
Conjunto adequado ao critério **MC/DC**.
- `test_hash_edge_pair.py`
Conjunto adequado ao critério **edge-pair**.
- `test_hash_prime_paths.py`
Conjunto adequado ao critério **prime paths**.

Regras para os arquivos de teste

Cada arquivo deve conter:

- apenas os casos de teste destinados ao critério correspondente;
- testes executáveis automaticamente (unittest);
- código claro e organizado;
- nomes de métodos de teste que ajudem a identificar o comportamento exercitado.

Requisito de não redundância

Para cada um dos quatro conjuntos, vale a seguinte exigência:

Nenhum caso de teste pode ser removido sem alterar a cobertura do conjunto em relação ao critério para o qual ele foi construído.

Assim:

- em `test_hash_desvios.py`, nenhum teste pode ser removido sem diminuir a cobertura de arestas;
- em `test_hash_mcdc.py`, nenhum teste pode ser removido sem diminuir a cobertura MC/DC;
- em `test_hash_edge_pair.py`, nenhum teste pode ser removido sem diminuir a cobertura edge-pair;
- em `test_hash_prime_paths.py`, nenhum teste pode ser removido sem diminuir a cobertura prime paths.

Não basta atingir alta cobertura: o conjunto deve ser **enxuto**.

O que o aluno deve fazer

Parte 1: Estudo do programa

Analise o código da Tabela Hash e identifique:

- os principais fluxos de controle;
- decisões simples e compostas;
- laços;
- tratamentos especiais, se existirem;
- comportamentos que possam gerar requisitos não executáveis.

Parte 2: Levantamento dos requisitos de teste

Para cada critério, levante explicitamente os requisitos de teste correspondentes:

- arestas;
- pares de arestas;

- caminhos primos;
- combinações necessários para MC/DC.

Parte 3: Construção dos conjuntos

Projete quatro conjuntos de testes:

- um por critério;
- cada um adequado ao seu critério;
- cada um sem casos supérfluos.

Parte 4: Análise cruzada de cobertura

Depois de construir os conjuntos, analise **cada conjunto em relação a todos os critérios**.

Exemplo:

- o conjunto de arestas deve ser analisado quanto à cobertura de arestas, MC/DC, edge-pair e prime paths;
- o conjunto de MC/DC também deve ser analisado quanto aos quatro critérios;
- e assim por diante.

Você deverá entregar:

1. Código-fonte dos testes

Os quatro arquivos:

- `test_hash_desvios.py`
- `test_hash_mcdc.py`
- `test_hash_edge_pair.py`
- `test_hash_prime_paths.py`

2. Relatório

Um relatório em PDF contendo, no mínimo, as seções abaixo.

Estrutura esperada do relatório

1. Identificação

- nome do aluno;
- disciplina;
- data;
- nome do programa testado.

2. Requisitos não executáveis

Indique quais requisitos foram considerados **não executáveis**, explicando:

- qual é o requisito;
- por que ele não pode ser exercitado;
- qual trecho do programa justifica essa conclusão.

Essa seção é obrigatória.

3. Análise de cobertura cruzada

Monte uma análise mostrando, para cada conjunto, a cobertura obtida em **todos** os critérios.

Sugestão de tabela:

Conjunto de testes	Arestas	MC/DC	Edge-pair	Prime paths
test_hash_desvios.py	...	...	...	...
test_hash_mcdc.py	...	...	...	...
test_hash_edge_pair.py	...	...	...	...
test_hash_prime_paths.py	...	...	...	...

4. Conclusões

Comente:

- qual critério exigiu mais testes;
- qual foi mais difícil de satisfazer;
- onde surgiram mais requisitos não executáveis;
- o que indica a tabela de cobertura cruzada.
- com base na definição desses critérios, você poderia pensar num critério alternativo, baseado em cobertura estrutural do código?

Ferramentas a serem usadas:

- **pymcdc:**
pip install pymcdc
- **cfgcoverage:**
pip install cfgcoverage